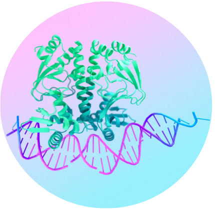
SciLifeLab

# AI agent collaboration with the Model Context Protocol (MCP)




Mahbub Ul Alam
Data Engineer (AI)
SciLifeLab Data Centre

Johan Alfredéen
Data Engineer (AI)
SciLifeLab Data Centre

Dinh Long Huynh
Doctoral Candidate
Uppsala University


09/06/26

# What you will build today...

- ❑ A drug discovery **MCP server** exposing tools
- ❑ A **MCP client** that discovers and calls tools dynamically
- ❑ **Security**: detect and defend against tool poisoning attacks
- ❑ **Guardrails**: input validation, blocked compounds, rate limiting
- ❑ **Integration**: connect your earlier **LangGraph agent** to MCP
- ❑ **Bonus 🎁: MCP Python SDK,** from basic to production
- ❑ **Bonus 🎁: integrating with real-world research works!**

# Outline: The bigger picture

## SESSION 2 — BUILDING THE MCP PIPELINE

**1**

### Raw Protocol

Understand MCP from scratch: JSON-RPC, resources, tools

Parts 1 – 2

READING

**2**

### Working Server

Build your own drug discovery MCP server & client

Part 3

HANDS-ON

**3**

### Safe Server

Add guardrails, security layer, and tool poisoning defence

Parts 4 – 5

HANDS-ON

**4**

### Connected Agent

LangGraph + MCP working together end-to-end

Part 6

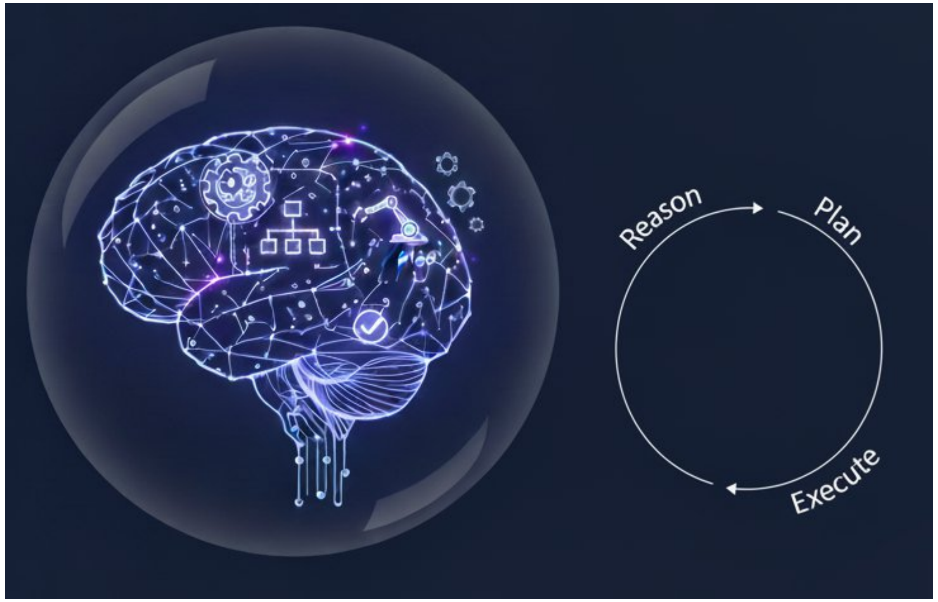
HANDS-ON

*By the end of this session, you will have built every stage of this diagram.*

[1]

# Our journey so far (session 1 recap)

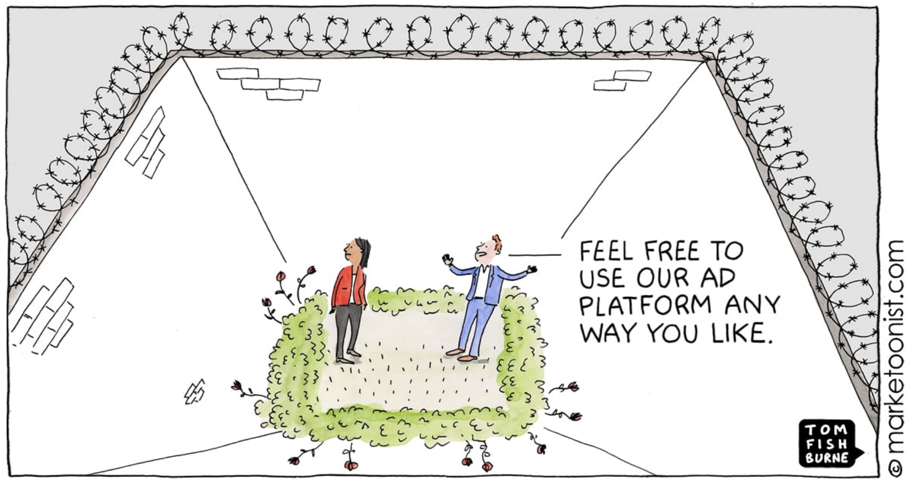
- In Session 1 you built a **LangGraph** drug discovery agent
- Three tools: **`resolve_smiles`**, **`get_properties`**, **`lit_search`**
- It can **reason, plan, and execute** a sequence of actions
- A working proof-of-concept for agentic AI in life sciences




[2]

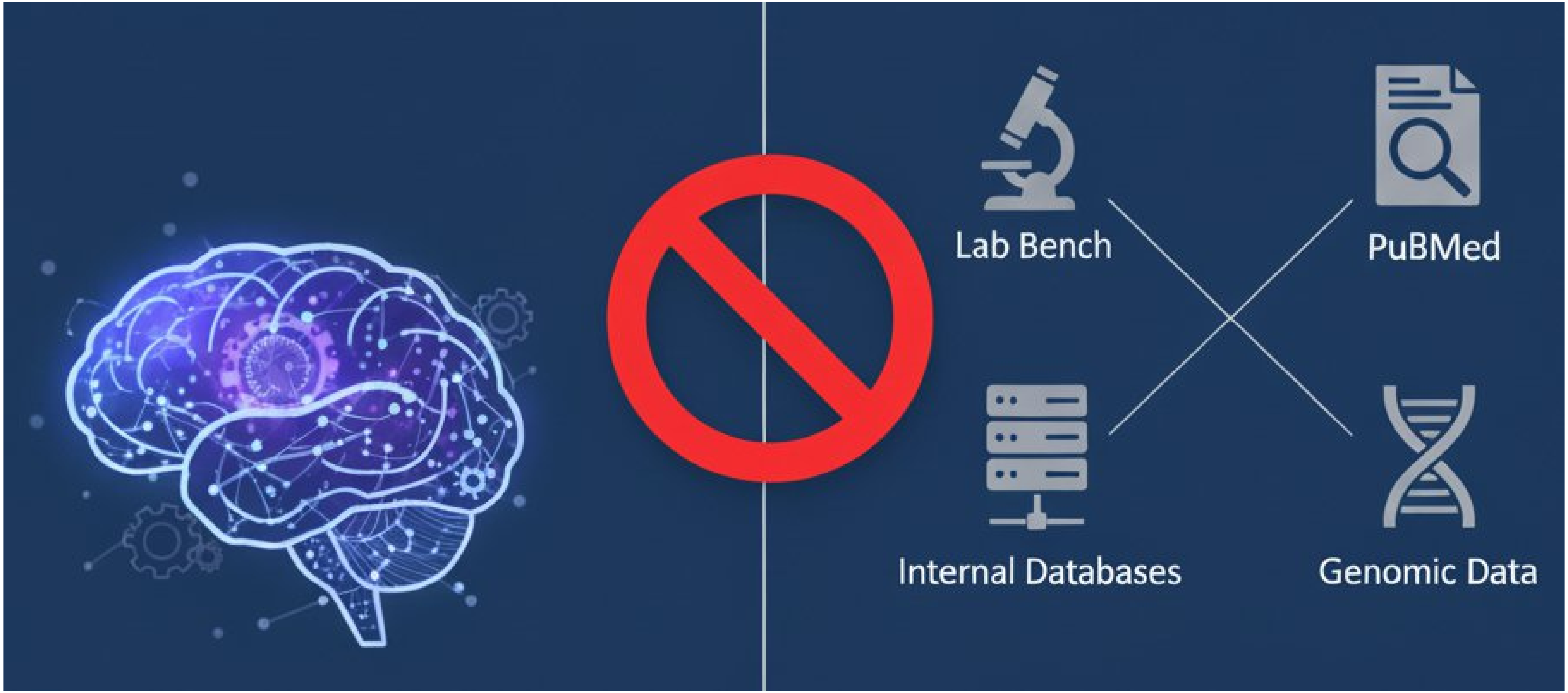
# But it was a walled garden!

- Your tools were **hardcoded** inside your Python file
- `resolve_smiles`, `get_properties`, and `lit_search` **only worked in your notebook**
- Any other agent or researcher had to **copy** your code **to use** them
- Perfect for a demo, but not for a real research environment




[3]

# The Real-world challenges: scalability & reinventing the wheel



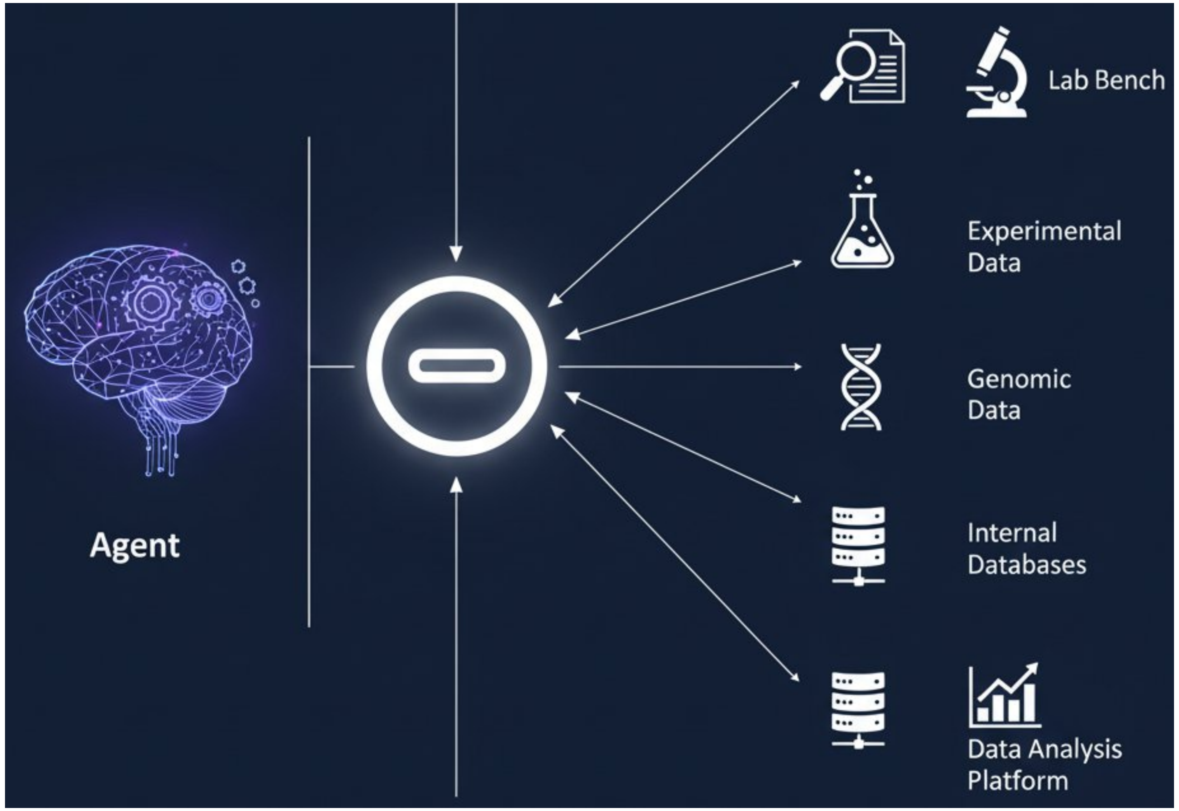

[2]

# The Real-world challenges: scalability & reinventing the wheel

- Need a **new tool** (e.g. ChEMBL, a private drug database)? → **Stop, code, redeploy**
- **Not flexible enough** for fast-paced drug discovery
- Every team builds **isolated agents** and **duplicates** effort
- Teams build their own agents and tools **independently**
- **Duplication, inconsistency, maintenance** burden
- **No standard way to share** or discover tools across groups

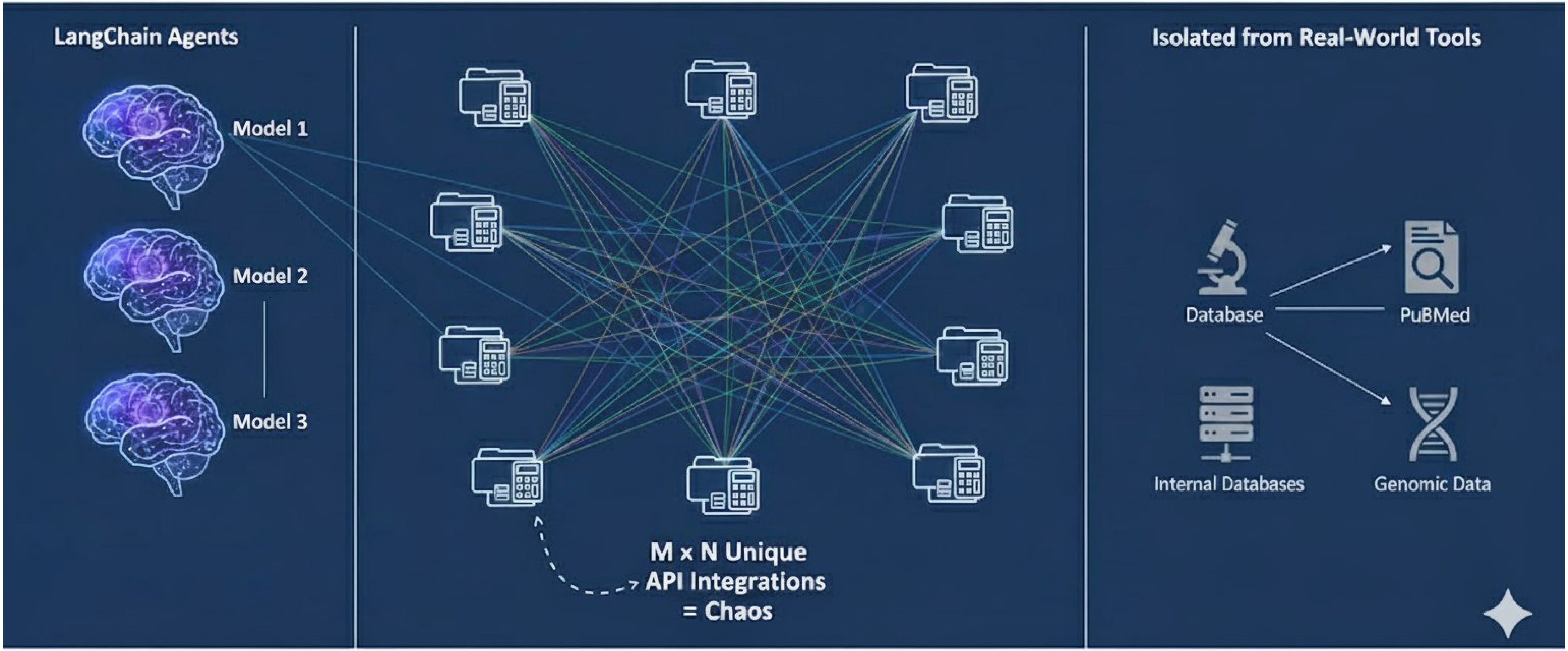
# Solution: introducing MCP (Model Context Protocol)!

- LangGraph = **The brain** (reasoning and planning)
- MCP = **The nervous system** (standardised connections)
- **Dynamic tool discovery**: agents find and use tools automatically
- **Unified ecosystem**: teams build and share tools composably



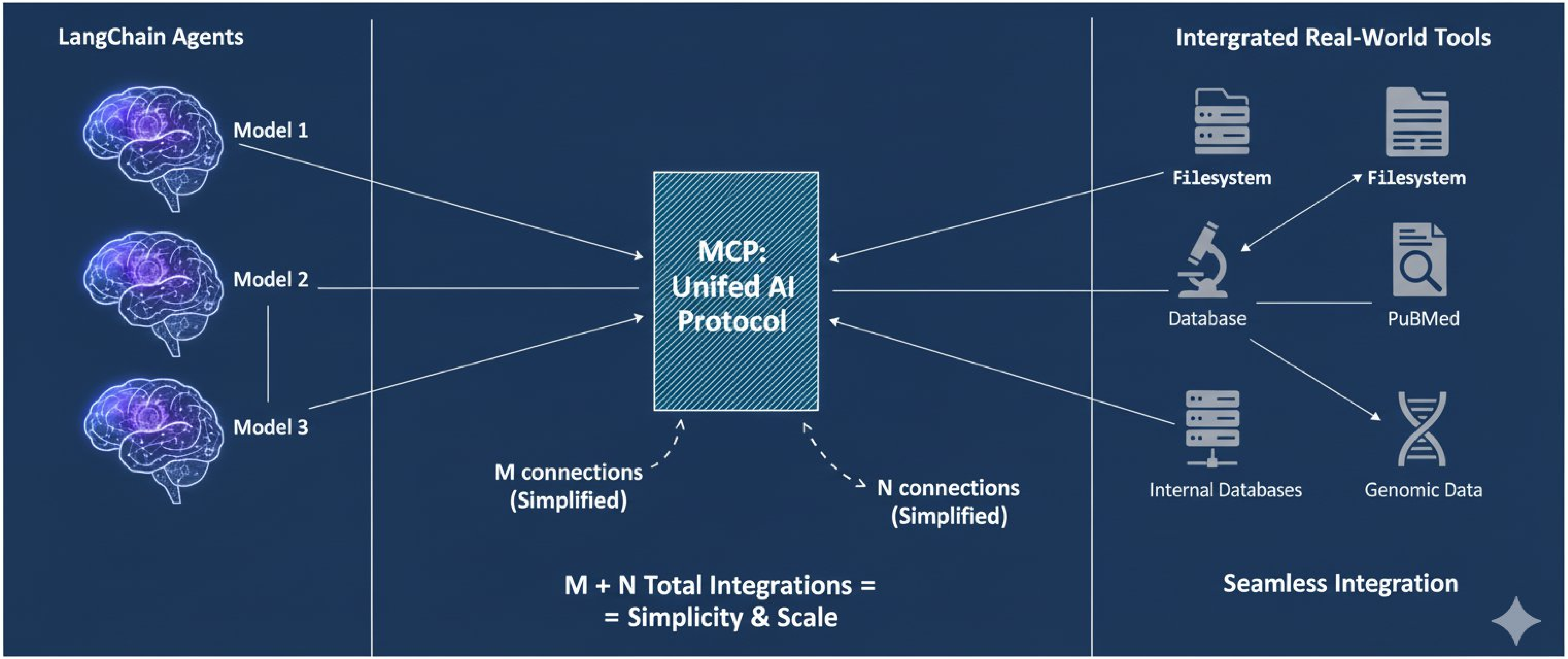

MCP: The **“USB-C for AI”** [2]

# The M×N integration problem



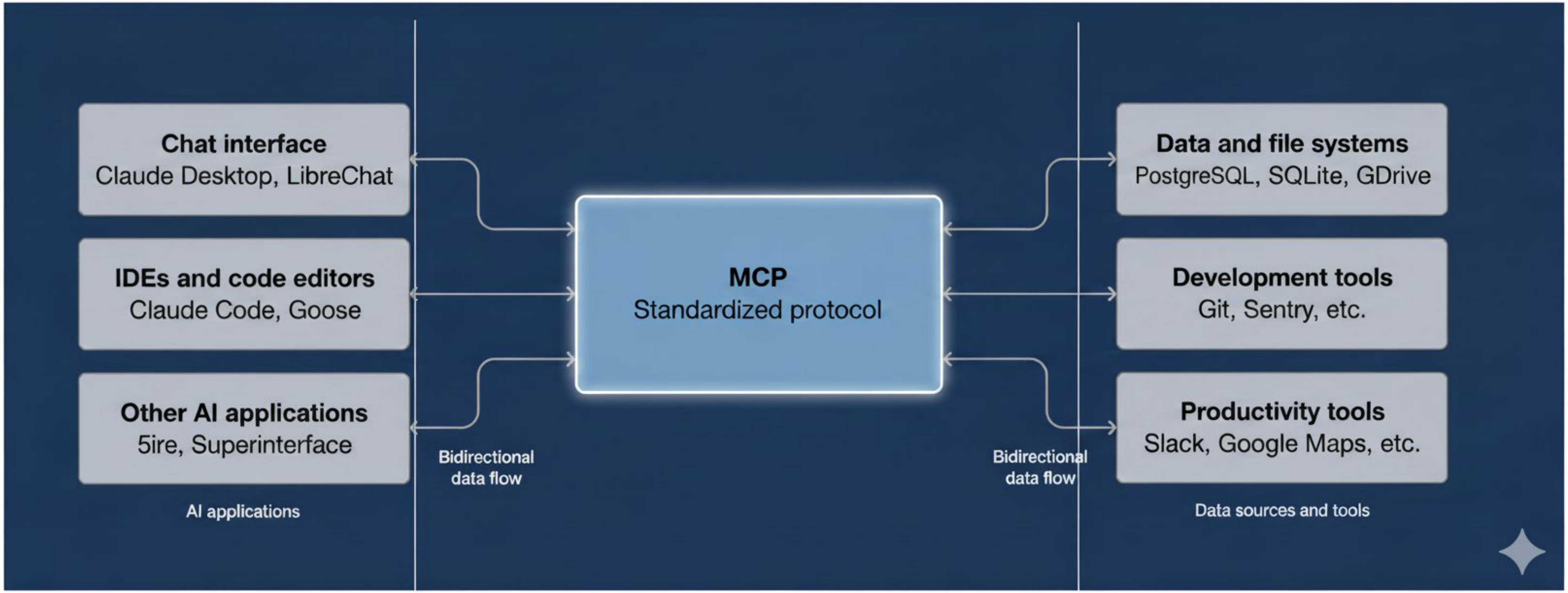

[4]

# With MCP (M+N solution)



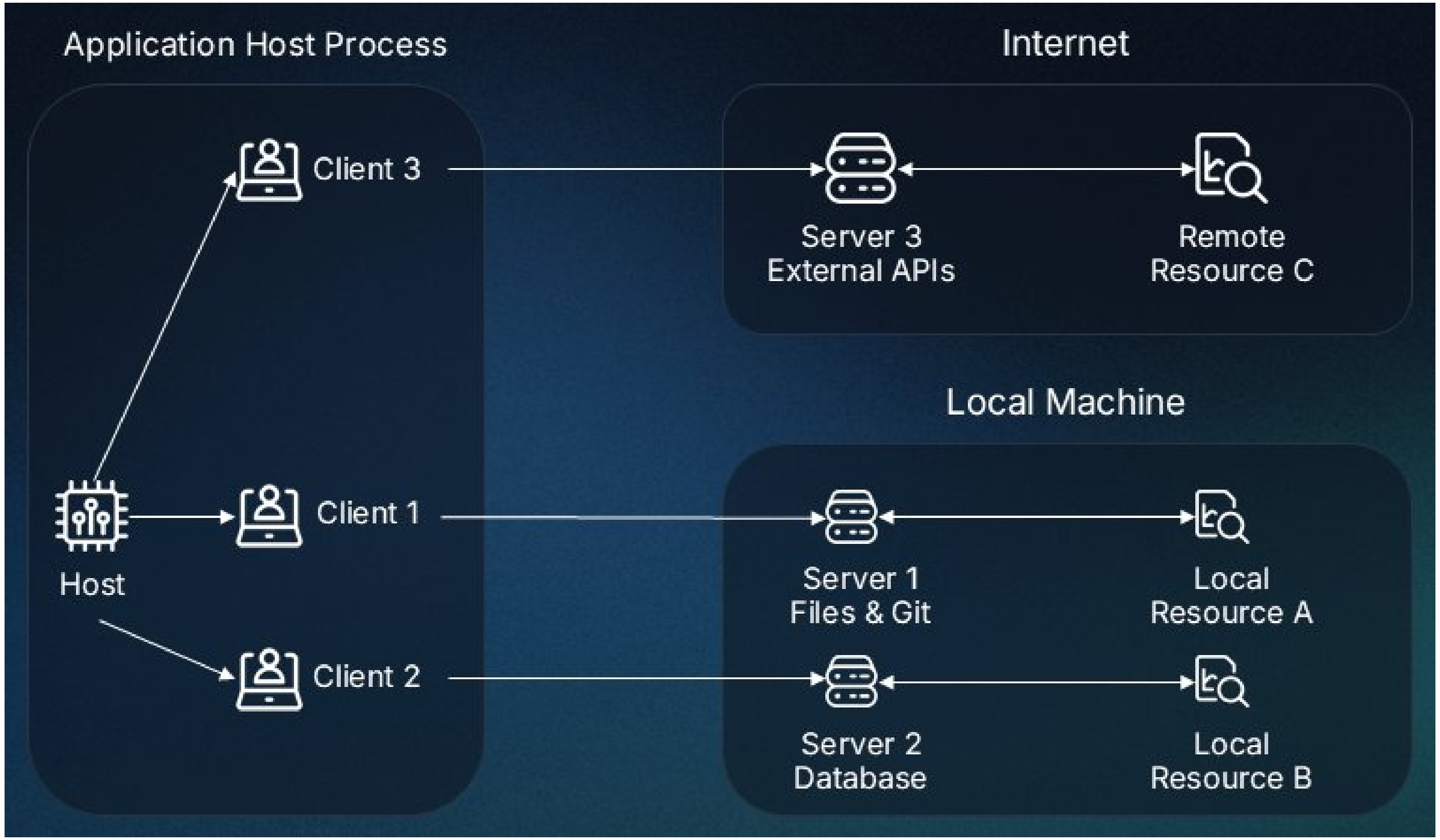

[4]

# MCP: overview




[5]

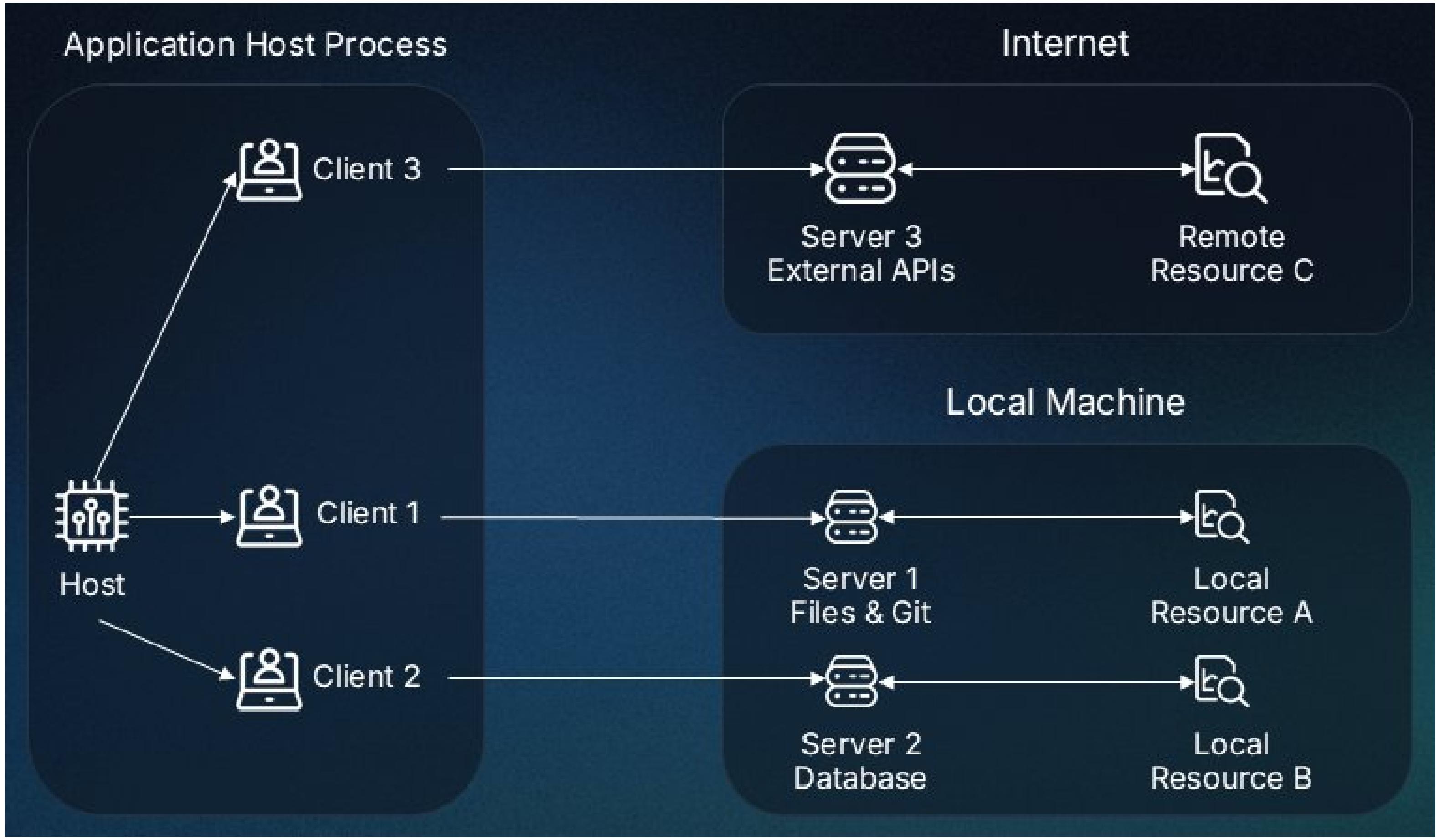
# MCP components: Host




[6]

**Host**: User-facing AI app that interacts with end-users and orchestrates requests, LLM processing, and external tools.

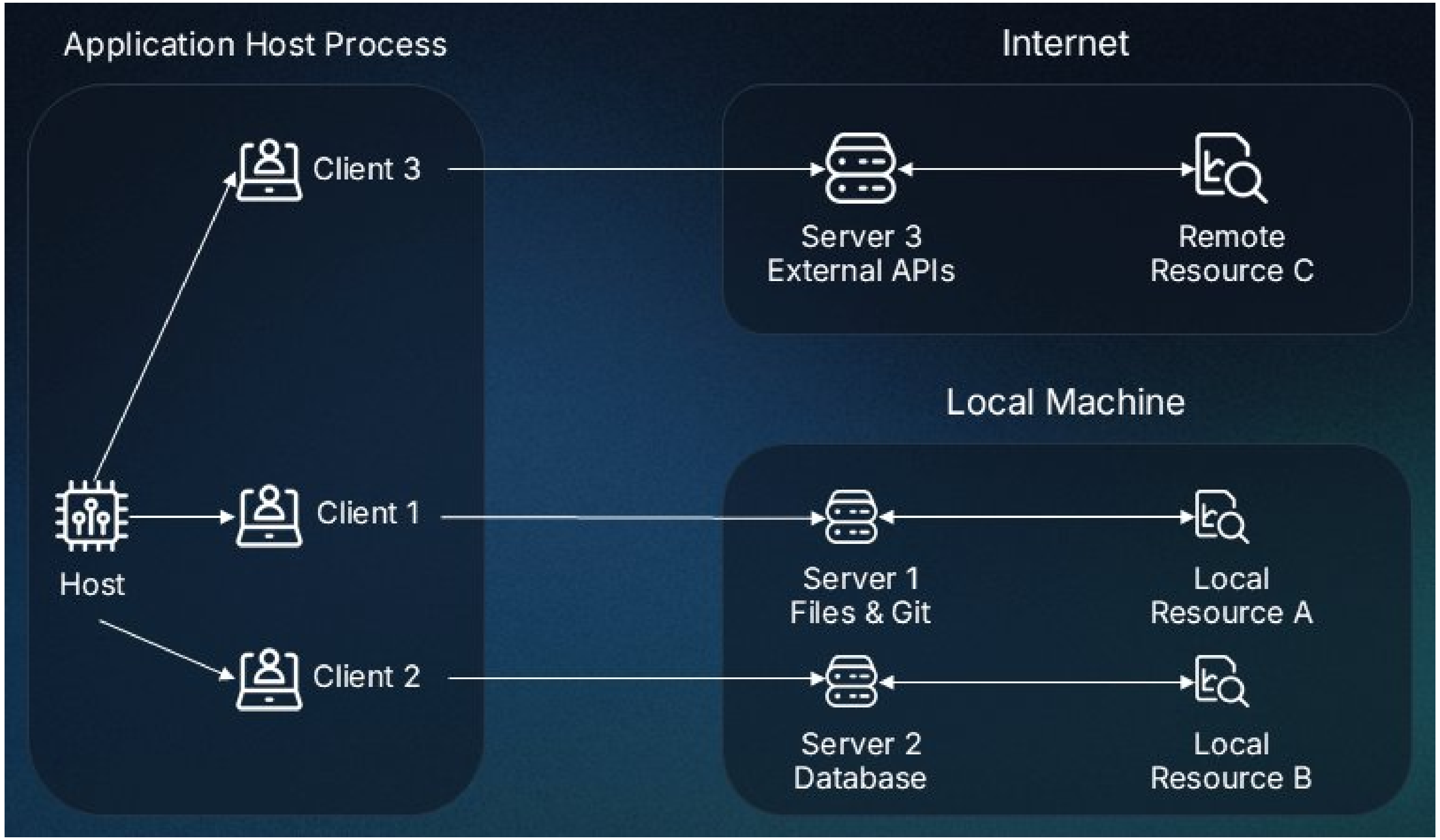
# MCP components: Client




[6]

**Client**: Manages communication between the Host and a single MCP Server, handling protocol details.

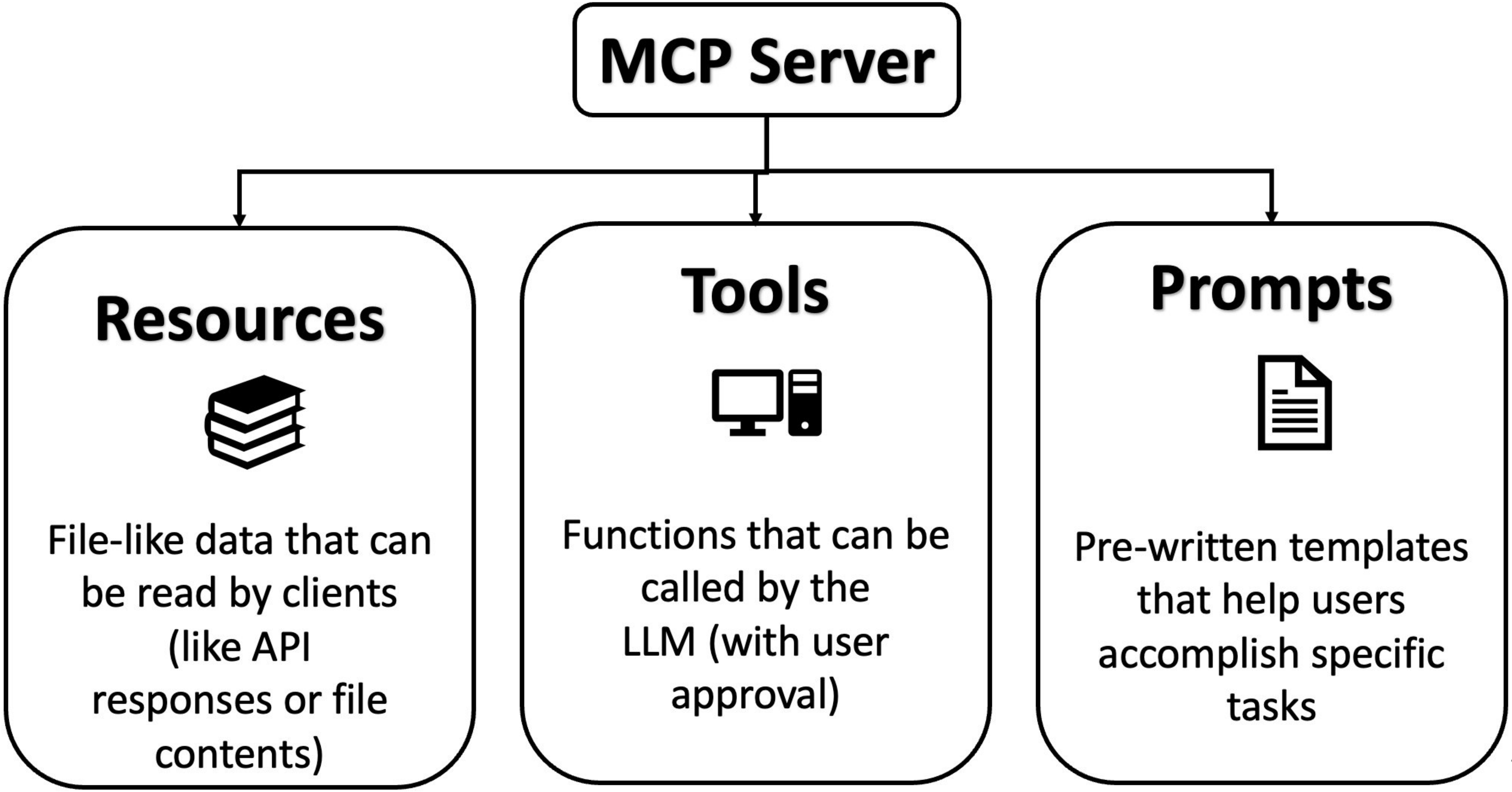
# MCP components: Server




[6]

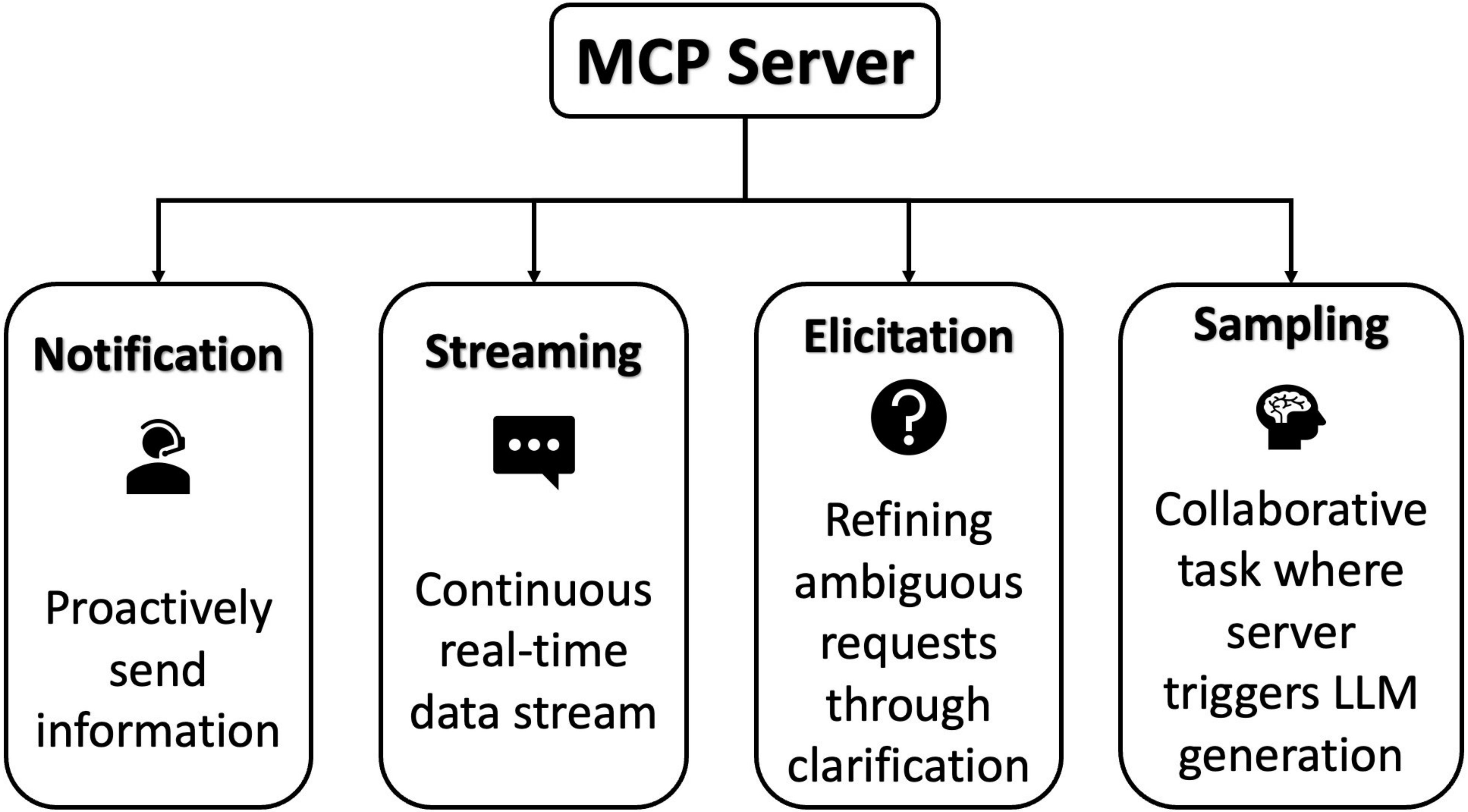
**Server**: External service exposing tools, resources, or prompts via the MCP protocol.

# MCP capabilities: basic



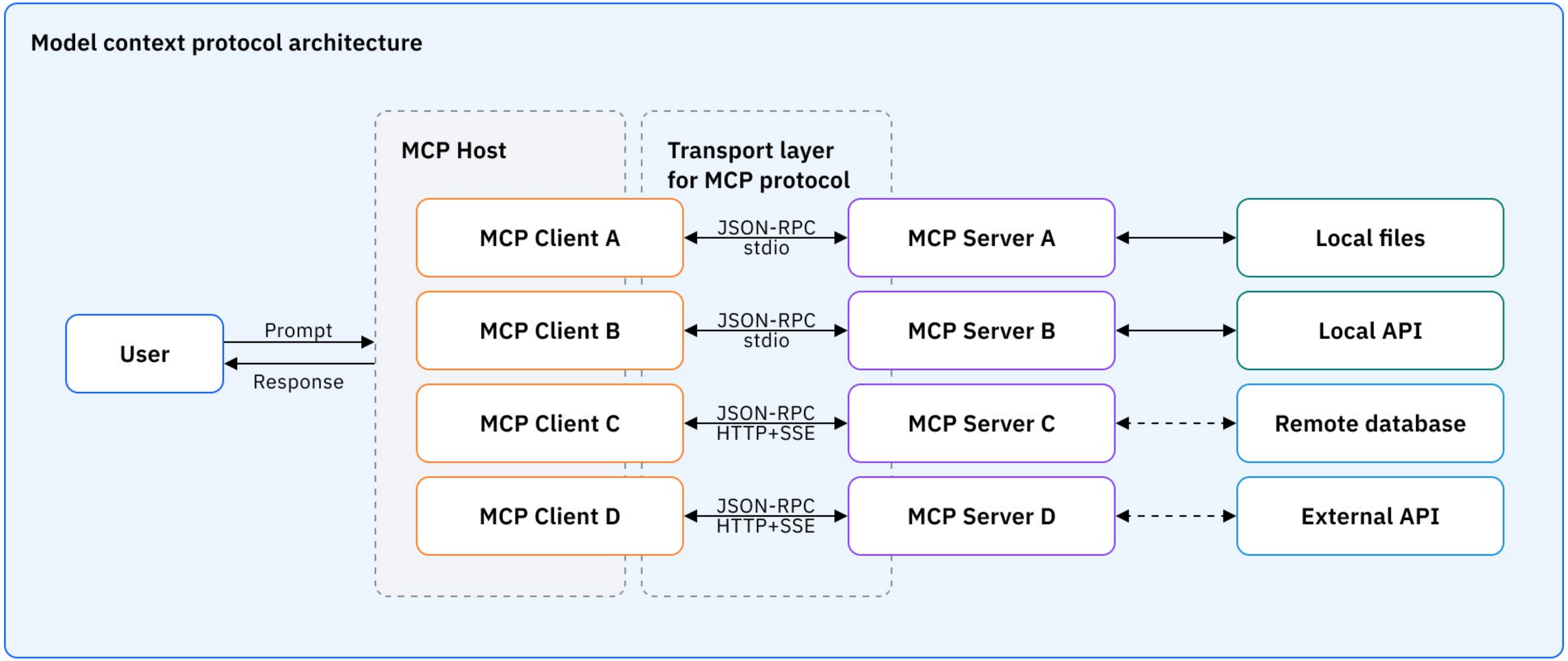

[7]

# MCP capabilities: advanced




[7]

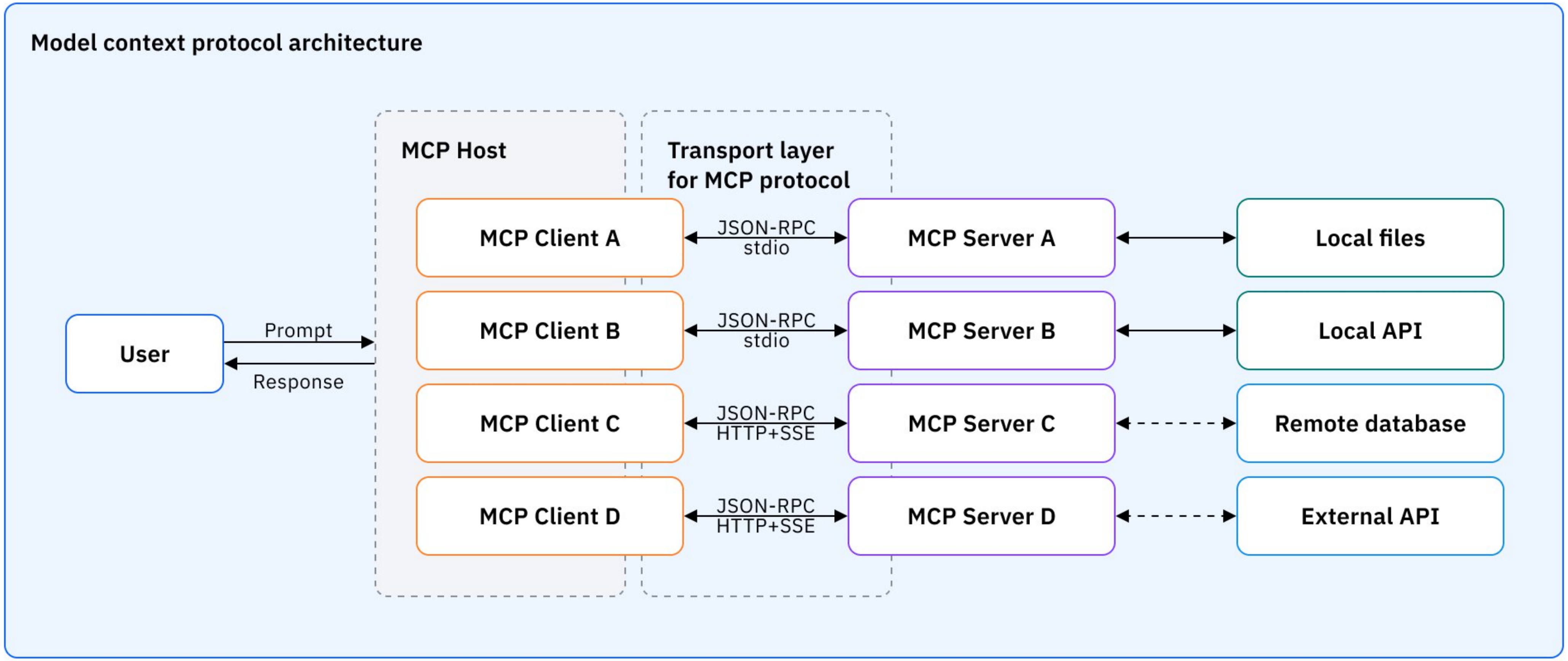
# MCP layers: data layer




[8]

**Data layer:** JSON-RPC protocol defining lifecycle management, tools, resources, prompts, and notifications.

# MCP layers: transport layer




[8]

**Transport layer:** Communication channels, connection handling, message framing, and authentication.

# Why security matters in MCP

- Tool descriptions are read by the LLM and **directly influence** its decisions
- In an open MCP ecosystem you may connect to servers **you didn't write**
- A crafted description is indistinguishable from a legitimate one at the protocol level, this is called **tool poisoning**

# MCP Security Hands On: Tool poisoning & Guardrails

- In **Part 5.3** you will build this **tool poisoning** attack and then defend against it

- **Guardrails** you will implement:
  - ✓ SMILES validation
  - ✓ blocked compounds
  - ✓ rate limiting
  - ✓ description sanitisation

# Bonus 🎁 MCP Python SDK: from basic to production

**Zero boilerplate:** decorators replace manual JSON-RPC handling

**One-line server startup:** mcp.run() vs. custom HTTP/SSE plumbing

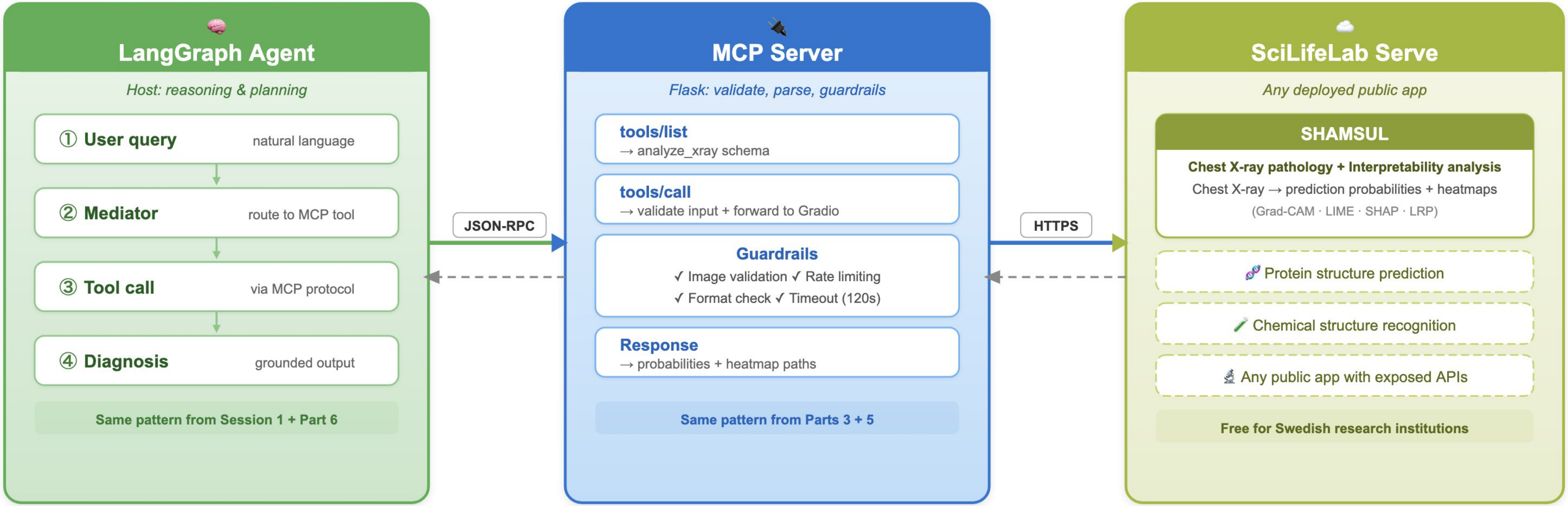
**Built-in LangChain bridge:** no adapter code required

**Type-safe by default:** Pydantic validation out of the box

**SDK speed:** same drug discovery agent, faster implementation

# Bonus 🎁 : SciLifeLab Serve Public App + MCP !

FROM WORKSHOP TO REAL WORLD




**Same MCP pattern you built today — works with any app on SciLifeLab Serve**

serve.scilifelab.se/apps · Bonus_Gradio_MCP/ in the workshop repo

[1]

# Next steps

**shorturl.at/rmCjV**

**Scan or use this link!**

- Make sure your **API key is ready**
- Start and explore! 🏝

# Additional resources

**Official MCP documentation**

https://modelcontextprotocol.io/docs/getting-started/intro

**A nice blog post**

https://www.ramotion.com/blog/mcp-protocol/

**Hugging Face course**

https://huggingface.co/learn/mcp-course/en/unit1/introduction

# References


[1] - AI-generated by Mahbub using Claude Code, Opus 4.6 Extended

[2] - AI-generated by Mahbub using Google Gemini 2.5 Pro

[3] - https://marketoonist.com/2022/05/walled-gardens.html

[4] - AI-generated by Mahbub using Google Gemini 2.5 Pro.
Original idea adapted from https://huggingface.co/learn/mcp-course/en/unit1/key-concepts

[5] - https://modelcontextprotocol.io/docs/getting-started/intro

[6] - https://medium.com/@sathishsuraj/understanding-the-model-context-protocol-mcp-architecture-implementation-and-use-cases-499f1e72673b

[7] - Original idea adapted from https://www.inferless.com/learn/how-to-connect-everyday-tools-with-mcp

[8] - https://www.ibm.com/think/podcasts/mixture-of-experts/ai-action-plan-chatgpt-agents-deepmind-imo

# Outline: The bigger picture

SESSION 2 — BUILDING THE MCP PIPELINE

### 1 Raw Protocol
Understand MCP from scratch: JSON-RPC, resources, tools

Parts 1 – 2

READING

### 2 Working Server
Build your own drug discovery MCP server & client

Part 3

HANDS-ON

### 3 Safe Server
Add guardrails, security layer, and tool poisoning defence

Parts 4 – 5

HANDS-ON

### 4 Connected Agent
LangGraph + MCP working together end-to-end

Part 6

HANDS-ON

*You have just built every stage of this diagram. The walled garden is open.*

[1]